



Module 4: Clustering and Association Rule Mining

[13H]

Clustering algorithms (k-means, hierarchical clustering, DBSCAN), Density-based clustering (OPTICS, Mean Shift), Association rule mining (Apriori algorithm, FP-growth algorithm), evaluation metrics for clustering and association rule mining (silhouette coefficient, lift, confidence, support).

S. No	Topic	Page No
4.1	Clustering Algorithms	03
4.1.1	K means Clustering	03
4.1.2	Hierarchical Clustering	06
4.1.3	DBSCAN Clustering - Density based clustering	12
4.2	Density Based Clustering	14
4.2.1	OPTICS Clustering - ML	14
4.2.2	Mean-Shift Clustering	17
4.3	Association Rule Mining	20
4.3.1	Apriori Algorithm	23
4.3.2	Frequent Pattern Growth Algorithm	29
4.4	Evaluation metrics for Clustering and Association Rule Mining	34
4.4.1	Silhouette Coefficient (Clustering Evaluation)	34
4.4.2	Support, Confidence, and Lift (Association Rule Mining)	36

Module – 4

Clustering and Association Rule Mining

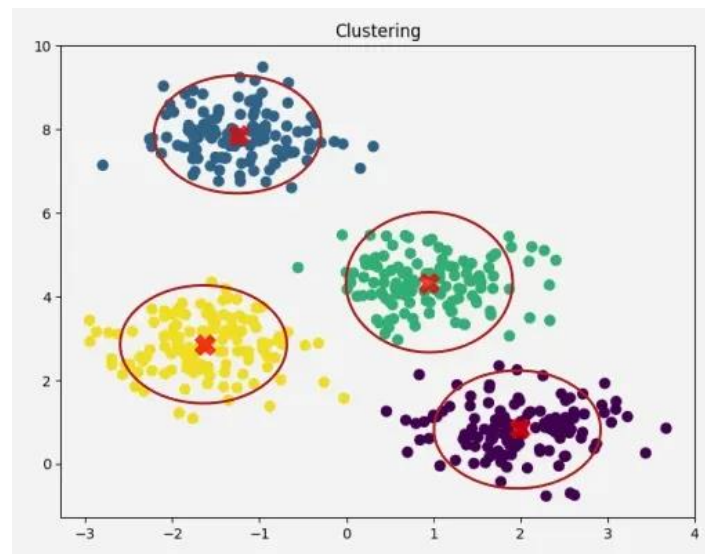
Clustering

Clustering is an unsupervised machine learning technique that groups similar data points together into clusters based on their characteristics, without using any labeled data. The objective is to ensure that data points within the same cluster are more similar to each other than to those in different clusters, enabling the discovery of natural groupings and hidden patterns in complex datasets.

- **Goal:** Discover the natural grouping or structure in unlabeled data without predefined categories.



- **How:** Data points are assigned to clusters based on similarity or distance measures.
- **Similarity Measures:** Can include Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.
- **Output:** Each group is assigned a cluster ID, representing shared characteristics within the cluster.



For example, if we have customer purchase data, clustering can group customers with similar shopping habits. These clusters can then be used for targeted marketing, personalized recommendations or customer segmentation.

Types of Clustering

Let's see the types of clustering,

1. Hard Clustering: In hard clustering, each data point strictly belongs to exactly one cluster, no overlap is allowed. This approach assigns a clear membership, making it easier to interpret and use for definitive segmentation tasks.

- **Example:** If clustering customer data into 2 segments, each customer belongs fully to either Cluster 1 or Cluster 2 without partial memberships.
- **Use cases:** Market segmentation, customer grouping, document clustering.
- **Limitations:** Cannot represent ambiguity or overlap between groups; boundaries are crisp.



2. Soft Clustering: Soft clustering assigns each data point a probability or degree of membership to multiple clusters simultaneously, allowing data points to partially belong to several groups.

- **Example:** A data point may have a 70% membership in Cluster 1 and 30% in Cluster 2, reflecting uncertainty or overlap in group characteristics.
- **Use cases:** Situations with overlapping class boundaries, fuzzy categories like customer personas or medical diagnosis.
- **Benefits:** Captures ambiguity in data, models gradual transitions between clusters.

4.1 Clustering Algorithms

4.1.1 K means Clustering

K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression, and pattern discovery
- Useful when you need structure from raw, unorganized data

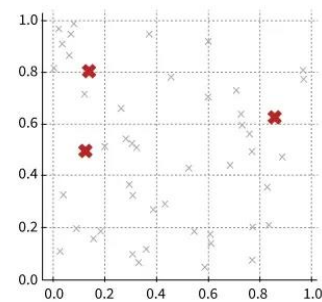
Working of K-Means Clustering

Suppose we are given a data set of items with certain features and values for these features like a vector. The task is to categorize those items into groups. To achieve this we will use the K-means algorithm. " k " represents the number of groups or clusters we want to classify our items into.

Choose Initial Centroids

Centroids are randomly chosen from the data points. These represent the initial cluster centers.

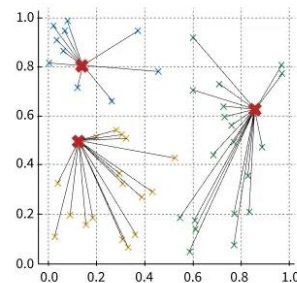
✱ Centroids ✕ Data Points



Assign Points to Nearest Centroid

Each point is assigned to the nearest centroid, forming clusters

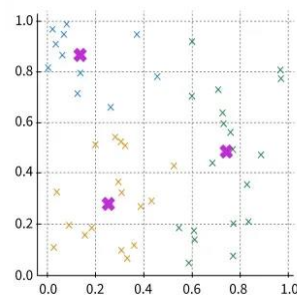
✱ Centroids ✕ Cluster 1
 ✕ Cluster 2 ✕ Cluster 3



Update Centroids

Centroids are recalculated as the mean of the points in each cluster

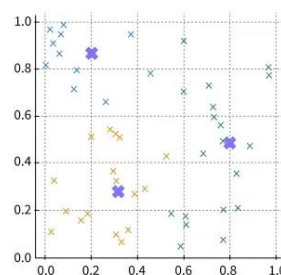
✱ New Centroids
 ✕ Cluster 1
 ✕ Cluster 2
 ✕ Cluster 3



Repeat Until Convergence

This process repeats until the centroids stabilize and do not move further.

✱ Final Centroids
 ✕ Cluster 1
 ✕ Cluster 2
 ✕ Cluster 3





The algorithm will categorize the items into " k " groups or clusters of similarity. To calculate that similarity we will use the Euclidean distance as a measurement. The algorithm works as follows:

1. **Initialization:** We begin by randomly selecting k cluster centroids.
2. **Assignment Step:** Each data point is assigned to the nearest centroid, forming clusters.
3. **Update Step:** After the assignment, we recalculate the centroid of each cluster by averaging the points within it.
4. **Repeat:** This process repeats until the centroids no longer change or the maximum number of iterations is reached.

The goal is to partition the dataset into k clusters such that data points within each cluster are more similar to each other than to those in other clusters.

Why Use K-Means Clustering?

K-Means is popular in a wide variety of applications due to its simplicity, efficiency and effectiveness. Here's why it is widely used:

1. **Data Segmentation:** One of the most common uses of K-Means is segmenting data into distinct groups. For example, businesses use K-Means to group customers based on behavior, such as purchasing patterns or website interaction.
2. **Image Compression:** K-Means can be used to reduce the complexity of images by grouping similar pixels into clusters, effectively compressing the image. This is useful for image storage and processing.
3. **Anomaly Detection:** K-Means can be applied to detect anomalies or outliers by identifying data points that do not belong to any of the clusters.
4. **Document Clustering:** In natural language processing (NLP), K-Means is used to group similar documents or articles together. It's often used in applications like recommendation systems or news categorization.
5. **Organizing Large Datasets:** When dealing with large datasets, K-Means can help in organizing the data into smaller, more manageable chunks based on similarities, improving the efficiency of data analysis.

Challenges with K-Means Clustering

K-Means algorithm has the following limitations:

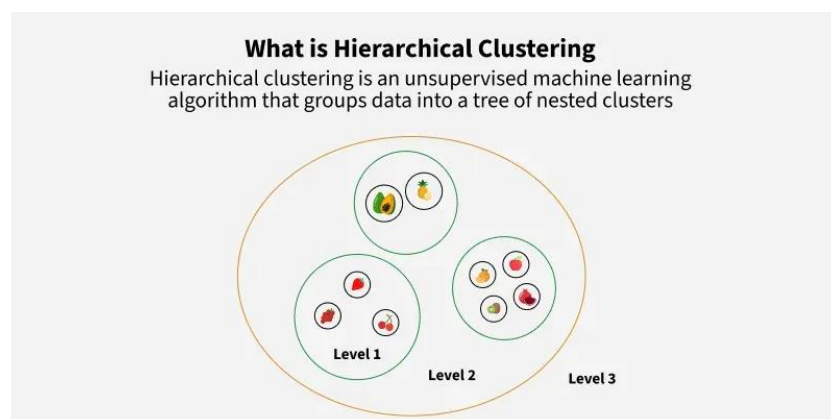


- **Choosing the Right Number of Clusters (k):** One of the biggest challenges is deciding how many clusters to use.
- **Sensitive to Initial Centroids:** The final clusters can vary depending on the initial random placement of centroids.
- **Non-Spherical Clusters:** K-Means assumes that the clusters are spherical and equally sized. This can be a problem when the actual clusters in the data are of different shapes or densities.
- **Outliers:** K-Means is sensitive to outliers, which can distort the centroid and, ultimately, the clusters.

4.1.2 Hierarchical Clustering

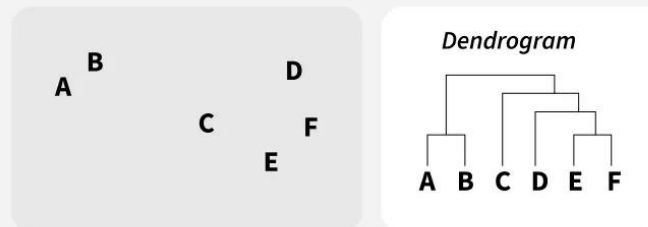
Hierarchical Clustering is an unsupervised learning technique that groups data into a hierarchy of clusters based on similarity. It builds a tree-like structure (dendrogram) that helps visualize relationships and decide the optimal number of clusters.

- Does not require pre-selecting the number of clusters.
- Uses agglomerative or divisive approaches.
- Commonly applied in data exploration and pattern discovery.
- It is commonly used in pattern recognition, customer segmentation and image grouping.

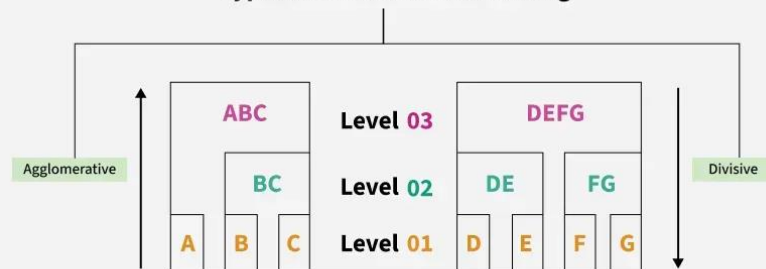


What is a Dendrogram?

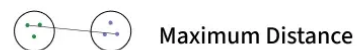
A **dendrogram** is a tree that shows how clusters are merged step-by-step. We **cut** the dendrogram at a certain height to form final clusters.



Types of Hierarchical Clustering



Distance Metrics in Hierarchical Clustering





Imagine we have four fruits with different weights: an apple (100g), a banana (120g), a cherry (50g) and a grape (30g). Hierarchical clustering starts by treating each fruit as its own group.

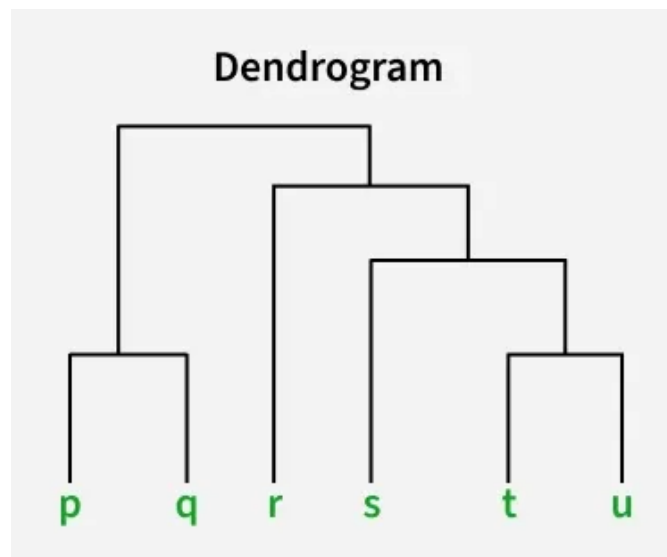
- Start with each fruit as its own cluster.
- Merge the closest items: grape (30g) and cherry (50g) are grouped first.
- Next, apple (100g) and banana (120g) are grouped.
- Finally, these two clusters merge into one.

Finally all the fruits are merged into one large group, showing how hierarchical clustering progressively combines the most similar data points.

Dendrogram

A dendrogram is like a family tree for clusters. It shows how individual data points or groups of data merge together. The bottom shows each data point as its own group and as we move up, similar groups are combined.

The lower the merge point, the more similar the groups are. It helps us see how things are grouped step by step.



- At the bottom of the dendrogram the points P, Q, R, S and T are all separate.
- As we move up, the closest points are merged into a single group.
- The lines connecting the points show how they are progressively merged based on similarity.
- The height at which they are connected shows how similar the points are to each other; the shorter the line the more similar they are

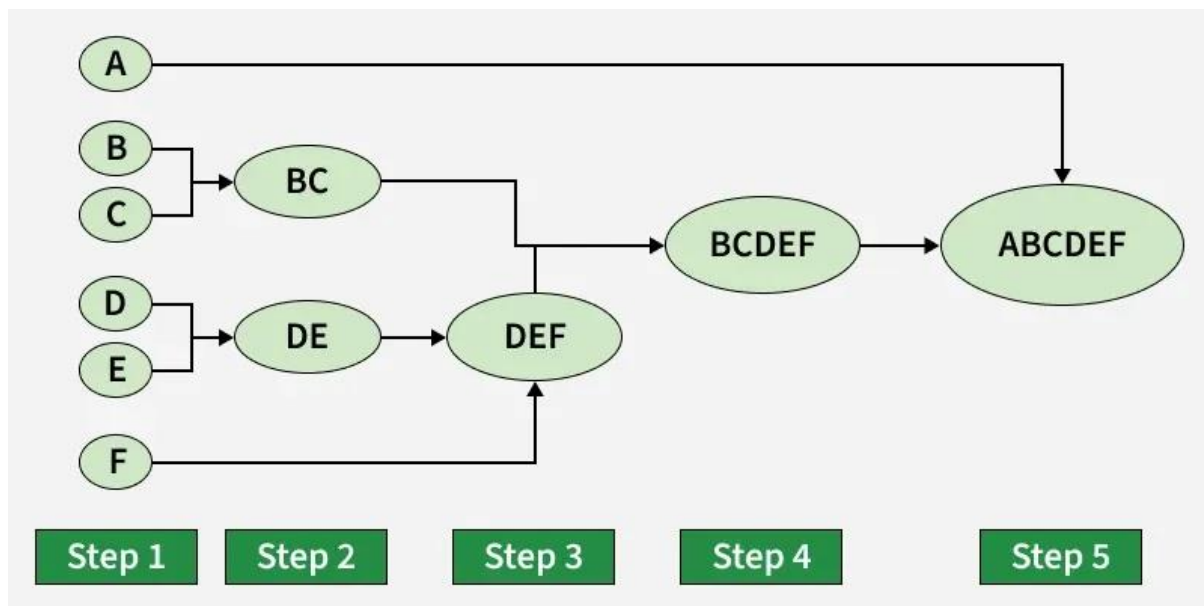
Types of Hierarchical Clustering

Now we understand the basics of hierarchical clustering. There are two main types of hierarchical clustering.

1. Agglomerative Clustering
2. Divisive clustering

1. Hierarchical Agglomerative Clustering

It is also known as the bottom-up approach or Hierarchical Agglomerative Clustering (HAC). Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerate pairs of clusters until all clusters have been merged into a single cluster that contains all data.



Workflow for Hierarchical Agglomerative clustering

1. **Start with individual points:** Each data point is its own cluster. For example if we have 5 data points we start with 5 clusters each containing just one data point.
2. **Calculate distances between clusters:** Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters:** Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix:** After merging we now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.
5. **Repeat steps 3 and 4:** Keep merging the closest clusters and updating the distance matrix until we have only one cluster left.
6. **Create a dendrogram:** As the process continues we can visualize the merging of clusters using a tree-like diagram called a dendrogram. It shows the hierarchy of how clusters are merged.

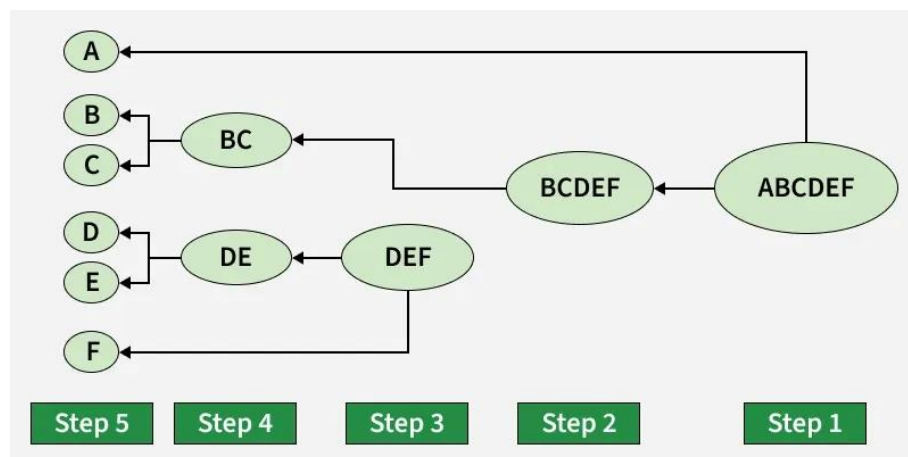


2. Hierarchical Divisive clustering

Divisive Clustering is also known as a top-down approach. Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been split into singleton clusters.

Workflow for Hierarchical Divisive clustering :

1. **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
2. **Split the cluster:** Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
3. **Repeat the process:** For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.
4. **Stop when each data point is in its own cluster:** Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.



Computing Distance Matrix

While merging two clusters we check the distance between two every pair of clusters and merge the pair with the least distance/most similarity. But the question is how is that distance determined. There are different ways of defining Inter Cluster distance/similarity. Some of them are:

- **Min Distance:** Find the minimum distance between any two points of the cluster.
- **Max Distance:** Find the maximum distance between any two points of the cluster.
- **Group Average:** Find the average distance between every two points of the clusters.
- **Ward's Method:** The similarity of two clusters is based on the increase in squared error when two clusters are merged.



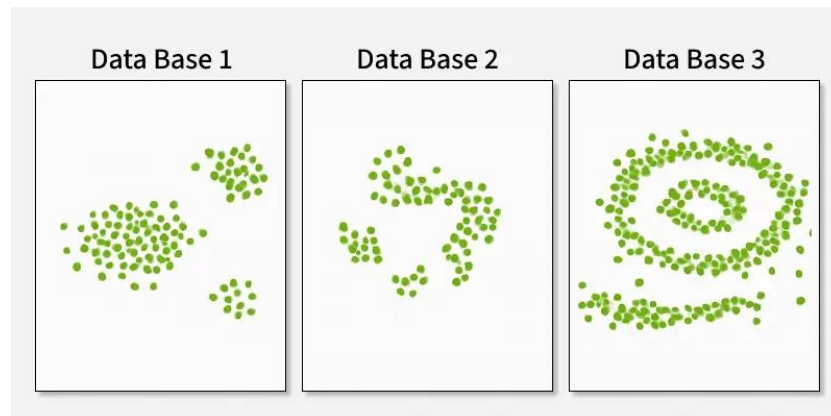
The image compares cluster distance methods:

- Min uses the shortest distance between clusters
- Max uses the longest
- Group Average computes the mean of all pairwise distances
- Ward's method minimizes the increase in within-cluster variance during merging

4.1.3 DBSCAN Clustering - Density based clustering

DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space. It identifies clusters as dense regions in the data space separated by areas of lower density. Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:

- **Arbitrary-Shaped Clusters:** Clusters can take any shape not just circular or convex.
- **Noise and Outliers:** It effectively identifies and handles noise points without assigning them to any cluster.



The figure above shows a data set with clustering algorithms: K-Means and Hierarchical handling compact, spherical clusters with varying noise tolerance while DBSCAN manages arbitrary-shaped clusters and noise handling.

Key Parameters in DBSCAN

1. eps: This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps they are considered neighbors. A common method to determine eps is by analyzing the k-distance graph. Choosing the right eps is important:

- If eps is too small most points will be classified as noise.
- If eps is too large clusters may merge and the algorithm may fail to distinguish between them.

2. MinPts: This is the minimum number of points required within the **eps** radius to form a dense region. A general rule of thumb is to set $\text{MinPts} \geq D+1$ where **D** is the number of dimensions in the dataset.

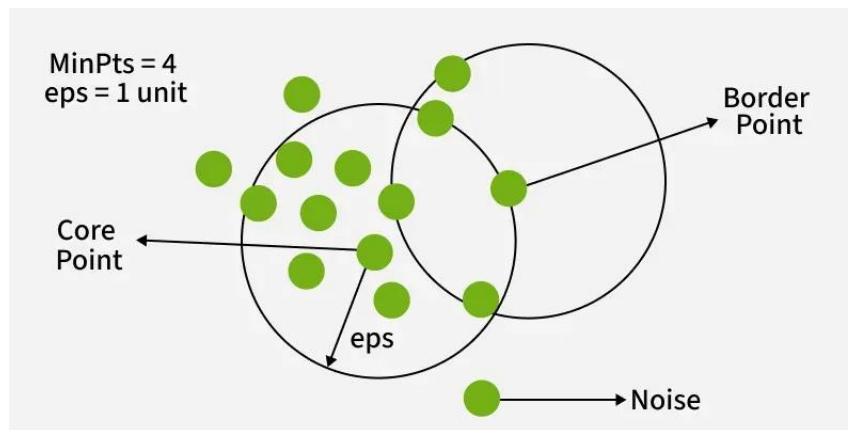
*For most cases a minimum value of **MinPts** = 3 is recommended.*

How Does DBSCAN Work?

DBSCAN works by categorizing data points into three types:

1. Core points which have a sufficient number of neighbors within a specified radius (epsilon)
2. Border points which are near core points but lack enough neighbors to be core points themselves
3. Noise points which do not belong to any cluster.

By iteratively expanding clusters from core points and connecting density-reachable points, DBSCAN forms clusters without relying on rigid assumptions about their shape or size.



Steps in the DBSCAN Algorithm

1. **Identify Core Points:** For each point in the dataset count the number of points within its eps neighborhood. If the count meets or exceeds MinPts mark the point as a core point.
2. **Form Clusters:** For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the eps radius of the core point and add them to the cluster.
3. **Density Connectivity:** Two points a and b are density-connected if there exists a chain of points where each point is within the eps radius of the next and at least one point in the chain is a core point. This chaining process ensures that all points in a cluster are connected through a series of dense regions.
4. **Label Noise Points:** After processing all points any point that does not belong to a cluster is labeled as noise.



4.2 Density Based Clustering

4.2.1 OPTICS Clustering - ML

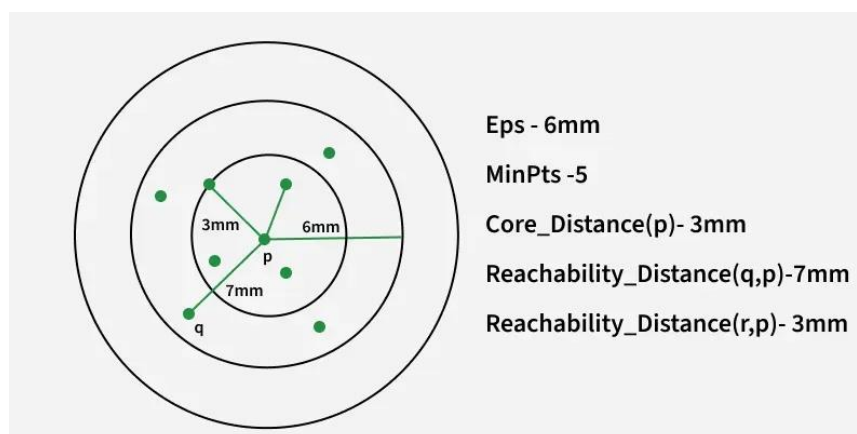
OPTICS (Ordering Points To Identify the Clustering Structure) is a density-based clustering algorithm similar to DBSCAN clustering. Unlike DBSCAN which struggles with varying densities, OPTICS does not directly assign clusters but instead creates a reachability plot which visually represents clusters. The key concepts in OPTICS are:

- **Core Distance:** The minimum distance needed for a point to be classified as a core point. If a point does not have enough nearby neighbours, its core distance is undefined.
- **Reachability Distance:** It is a measure of how difficult it is to reach from one point to another. It is calculated as the larger core distance of the starting point and the actual point.

Understanding Reachability Plot

A reachability plot is a graph that helps visualize clustering structures. It shows the reachability distance of each point in the dataset. It makes it ordered way based on how OPTICS processes them.

Here clusters appear as valleys in the plot where lower reachability distances indicate dense regions while peaks represent sparse regions or noise. To better understand the concept refer to the below image:





- Epsilon (Eps) = 6mm and MinPts = 5.
- The core distance of point p is 3mm meaning it needs at least 5 points within a 3mm radius to be considered as a core point.
- The reachability distance from q to p is 7mm (since q is farther than p's core distance).
- The reachability distance from r to p is 3mm (since r is within p's core distance).

It is more informative than DBSCAN as the reachability plot provides better understanding of clustering structure. Now we will learn about its working.

Working of OPTICS

1. The algorithm selects a starting point and checks if it has at least MinPts neighbors within Eps.
2. If the point meets the density requirement it is marked as a core point and nearby points are then analyzed.
3. Reachability distance is computed for each neighboring point.
4. Points are then processed in order of their reachability distance hence forming a reachability plot.
5. Clusters appear as valleys i.e low reachability distances and noise appears as peaks i.e high reachability distances.

OPTICS vs. DBSCAN Algorithm

We can compare OPTICS and DBSCAN to highlight their similarities and differences in clustering approach, flexibility and performance.

Feature	OPTICS	DBSCAN
Handles Varying Densities	Can detect clusters of different densities.	Struggles with varying densities as it requires a single epsilon value for all points.



Feature	OPTICS	DBSCAN
Cluster Identification	Uses a reachability plot for cluster extraction and identifies hierarchical structures.	Directly assigns clusters without hierarchical structure.
Hierarchical Structure	Yes it can detect nested clusters.	No it does not support hierarchical clustering.
Runtime Complexity	Higher due to sorting and ordering of reachability distances.	Lower as it processes fewer calculations.
Memory Cost	Requires more memory as it maintains a priority queue (Min Heap) for reachability.	Lower memory usage.
Fewer Parameters	Does not need to maintain the epsilon parameter hence reducing parameter tuning complexity.	Requires careful tuning of epsilon and minPts parameters for effective clustering.
Noise Handling	Does not directly identify noise points but high reachability distances may indicate noise.	Directly identify core points, boundary points and noise points.
Cluster Extraction	Produces a reachability distance plot for flexible extraction at different granularities.	Assigns clusters directly based on density criteria without additional plots.

OPTICS is widely used for clustering algorithm that works well for identifying clusters of varying densities. It provides flexibility through reachability plots which allows dynamic cluster extraction. While computationally more expensive it is useful for complex datasets where density variation is significant.

4.2.2 Mean-Shift Clustering



Meanshift is falling under the category of a clustering algorithm in contrast of Unsupervised learning that assigns the data points to the clusters iteratively by shifting points towards the mode (mode is the highest density of data points in the region, in the context of the Meanshift). As such, it is also known as the **Mode-seeking algorithm**. Mean-shift algorithm has applications in the field of image processing and computer vision.

Given a set of data points, the algorithm iteratively assigns each data point towards the closest cluster centroid and direction to the closest cluster centroid is determined by where most of the points nearby are at. So each iteration each data point will move closer to where the most points are at, which is or will lead to the cluster center. When the algorithm stops, each point is assigned to a cluster.

Unlike the popular K-Means cluster algorithm, mean-shift does not require specifying the number of clusters in advance. The number of clusters is determined by the algorithm with respect to the data.

Note: *The downside to Mean Shift is that it is computationally expensive $O(n^2)$.*

Mean-shift clustering is a non-parametric, density-based clustering algorithm that can be used to identify clusters in a dataset. It is particularly useful for datasets where the clusters have arbitrary shapes and are not well-separated by linear boundaries.

The basic idea behind mean-shift clustering is to shift each data point towards the mode (i.e., the highest density) of the distribution of points within a certain radius. The algorithm iteratively performs these shifts until the points converge to a local maximum of the density function. These local maxima represent the clusters in the data.

The process of mean-shift clustering algorithm can be summarized as follows:

Initialize the data points as cluster centroids.

Repeat the following steps until convergence or a maximum number of iterations is reached:

For each data point, calculate the mean of all points within a certain radius (i.e., the "kernel") centered at the data point.

Shift the data point to the mean.

Identify the cluster centroids as the points that have not moved after convergence.

Return the final cluster centroids and the assignments of data points to clusters.

One of the main advantages of mean-shift clustering is that it does not require the

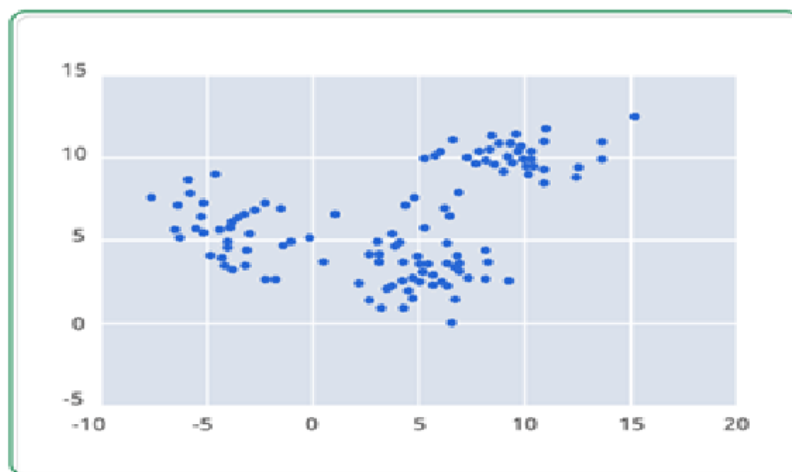


number of clusters to be specified beforehand. It also does not make any assumptions about the distribution of the data, and can handle arbitrary shapes and sizes of clusters. However, it can be sensitive to the choice of kernel and the radius of the kernel.

Mean-Shift clustering can be applied to various types of data, including image and video processing, object tracking and bioinformatics.

Kernel Density Estimation

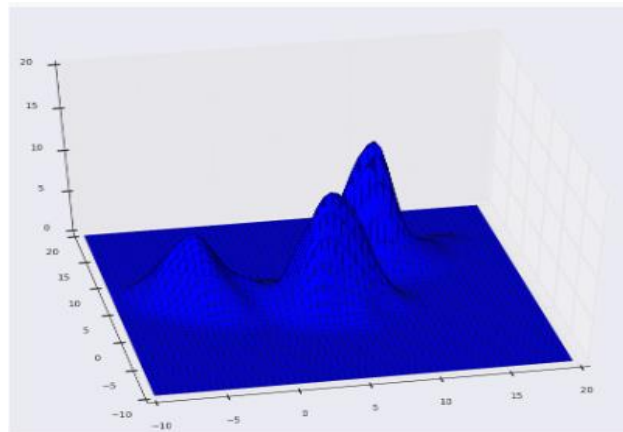
The first step when applying mean shift clustering algorithms is representing your data in a mathematical manner this means representing your data as points such as the set below.



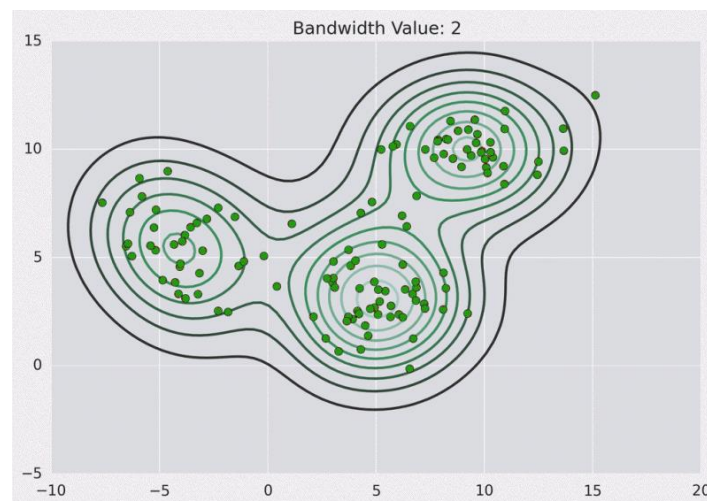
Mean-shift builds upon the concept of kernel density estimation, in short KDE. Imagine that the above data was sampled from a probability distribution. KDE is a method to estimate the underlying distribution also called the probability density function for a set of data. It works by placing a kernel on each point in the data set.

A kernel is a fancy mathematical word for a weighting function generally used in convolution. There are many different types of kernels, but the most popular one is the Gaussian kernel. Adding up all of the individual kernels generates a probability surface example density function. Depending on the kernel bandwidth parameter used, the resultant density function will vary. Below is the KDE surface for our points above using a Gaussian kernel with a kernel bandwidth of 2.

Surface plot:



Contour plot:



4.3 Association Rule Mining

Association Rule Mining is a powerful technique used to uncover meaningful relationships between variables within large datasets. They are designed to discover **“if-then” patterns**, providing insights into how data items are related and frequently occur together. These rules are particularly useful in identifying correlations and dependencies, enabling data-driven decision-making.

For instance, in a retail dataset, an association rule might identify that **“if a customer buys bread, they are likely to buy butter”**. Such insights help businesses improve cross-selling strategies, inventory management, and customer satisfaction.



Key Components of Association Rules

1. **Antecedent:** The “if” part of the rule, representing the condition.
 - Example: A customer buys bread.
2. **Consequent:** The “then” part of the rule, representing the outcome.
 - Example: The customer also buys butter.

Association rules are derived through algorithms that evaluate the frequency and strength of these relationships. They use metrics like **support**, **confidence**, and **lift** to measure the relevance and reliability of discovered patterns. These rules have applications in various fields, such as retail, healthcare, and marketing, where analyzing customer behavior or trends is critical for success.

Rule Evaluation Metrics

Association rules are evaluated using key metrics that determine their relevance, strength, and reliability. These metrics include **support**, **confidence**, and **lift**, which quantify the frequency and strength of relationships between data items.

1. Support

Support measures how frequently an itemset (both antecedent and consequent) appears in the dataset. It provides an indication of how common a particular association is.

Formula:

$$\text{Support} = \frac{\text{Transactions containing both antecedent and consequent}}{\text{Total transactions}}$$

Example: If bread and butter appear together in 100 out of 1,000 transactions, the support is:

$$\text{Support} = \frac{100}{1000} = 0.10 (10\%)$$

A higher support value indicates a more frequently occurring pattern in the dataset.



2. Confidence

Confidence measures the likelihood of the consequent occurring given that the antecedent has already occurred. It evaluates the reliability of the rule.

Formula:

$$\text{Confidence} = \frac{\text{Support of antecedent and consequent}}{\text{Support of antecedent}}$$

Example: If 70% of customers who buy bread also buy butter, the confidence is:

$$\text{Confidence} = 70\% = 0.70$$

Higher confidence suggests a stronger relationship between the antecedent and consequent.

3. Lift

Lift measures the strength of an association compared to its random occurrence in the dataset. It identifies how much more likely the antecedent and consequent are to appear together than independently.

Formula:

$$\text{Lift} = \frac{\text{Confidence}}{\text{Support of consequent}}$$

Example: A lift value greater than 1 indicates a strong positive association, while a value equal to 1 suggests no association. For instance, if the lift is 1.5, it means the antecedent makes the consequent 1.5 times more likely.

How Does Association Rule Learning Work?

Association rule learning is a multi-step process designed to identify meaningful patterns and relationships in large datasets. It involves two main stages:



1. **Identifying Frequent Itemsets:** The process begins by identifying frequent itemsets—combinations of items that appear together in transactions with a frequency above a predefined threshold. Metrics like support are used to measure how often these itemsets occur in the dataset. For example, a frequent itemset might reveal that bread and butter are purchased together in 10% of transactions.
2. **Generating Association Rules:** Once frequent itemsets are identified, association rules are generated. These rules take the form of if-then statements that describe relationships between items (e.g., “If a customer buys bread, they are likely to buy butter”). Metrics such as confidence and lift are applied to evaluate the strength and reliability of these rules.

Types of Association Rule Learning Algorithms

Several algorithms are used for association rule learning, each with unique strengths and applications. The three most commonly used algorithms are:

1. Apriori Algorithm

The **Apriori algorithm** employs a **breadth-first search** approach to identify frequent itemsets. It relies on the principle that all subsets of a frequent itemset must also be frequent, reducing the search space.

- **Advantage:** Simple to implement and effective for small datasets with low dimensionality.
- **Limitation:** Performance degrades significantly with large or dense datasets due to repeated scanning of the database.

2. Eclat Algorithm

The **Eclat algorithm** uses a **depth-first search** strategy to discover frequent itemsets. Instead of scanning the database multiple times, it represents transactions as vertical itemsets and directly computes intersections.

- **Advantage:** Efficient for datasets with **sparse data** or where there are fewer frequent itemsets.

3. FP-Growth Algorithm

The **FP-Growth (Frequent Pattern Growth)** algorithm leverages a **prefix-tree structure** called the **FP-tree** to represent transactional data compactly. Unlike Apriori, it avoids generating candidate itemsets explicitly, making it faster and more efficient.

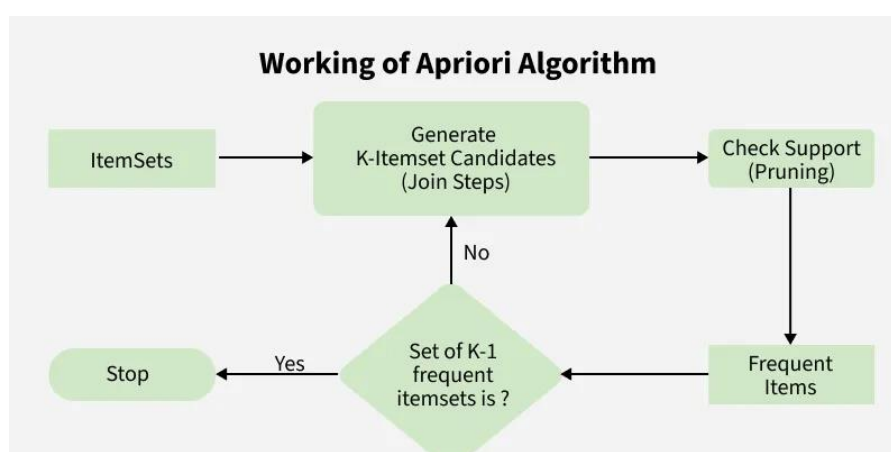
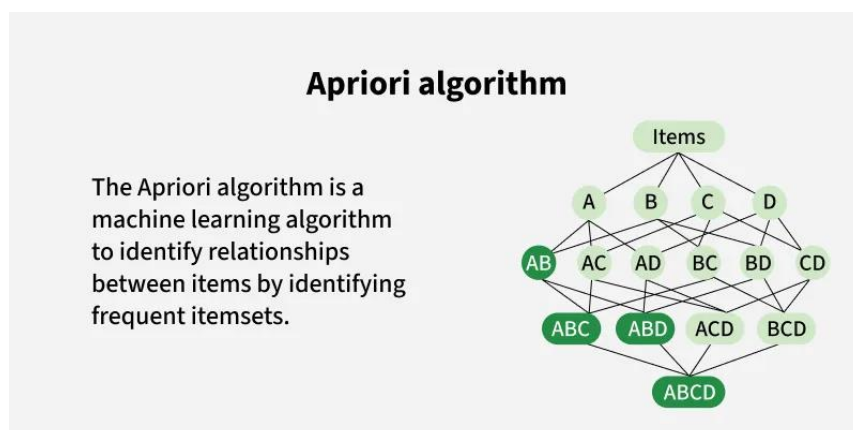


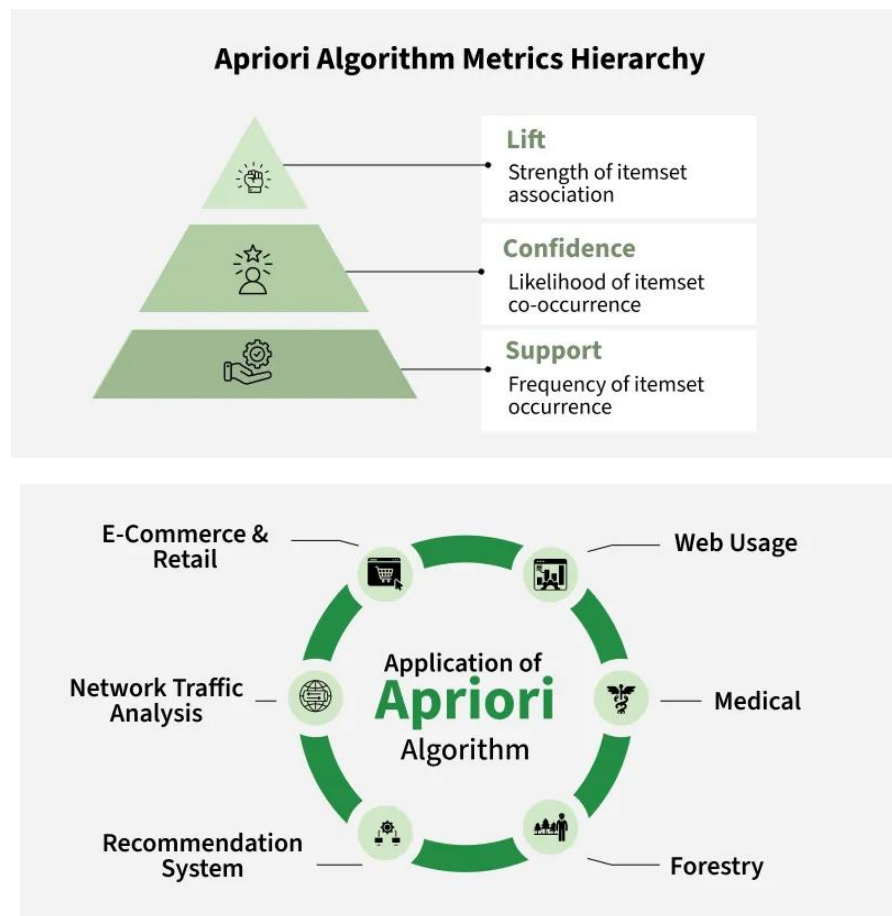
- **Advantage:** Significantly faster and more **memory-efficient** than Apriori, especially for large datasets.

4.3.1 Apriori Algorithm

Apriori Algorithm is a basic method used in data analysis to find groups of items that often appear together in large sets of data. It helps to discover useful patterns or rules about how items are related which is particularly valuable in market basket analysis.

Like in a grocery store if many customers buy bread and butter together, the store can use this information to place these items closer or create special offers. This helps the store sell more and make customers happy.





How the Apriori Algorithm Works?

The Apriori Algorithm operates through a systematic process that involves several key steps:

1. Identifying Frequent Item-Sets

- The Apriori algorithm starts by looking through all the data to count how many times each single item appears. These single items are called 1-Item-Sets.
- Next it uses a rule called minimum support this is a number that tells us how often an item or group of items needs to appear to be important. If an item appears often enough meaning its count is above this minimum support it is called a frequent Item-Set.

2. Creating Possible Item Group

- After finding the single items that appear often enough (frequent 1-item groups) the algorithm combines them to create pairs of items (2-item



groups). Then it checks which pairs are frequent by seeing if they appear enough times in the data.

- This process keeps going step by step making groups of 3 items, then 4 items and so on. The algorithm stops when it can't find any bigger groups that happen often enough.

3. Removing Infrequent Item Groups

- The Apriori algorithm uses a helpful rule to save time. This rule says: if a group of items does not appear often enough then any larger group that includes these items will also not appear often.
- Because of this, the algorithm does not check those larger groups. This way it avoids wasting time looking at groups that won't be important making the whole process faster.

4. Generating Association Rules

- The algorithm makes rules to show how items are related.
- It checks these rules using support, confidence and lift to find the strongest ones.

Key Metrics of Apriori Algorithm

- **Support:** This metric measures how frequently an item appears in the dataset relative to the total number of transactions. A higher support indicates a more significant presence of the Item-Set in the dataset. Support tells us how often a particular item or combination of items appears in all the transactions like Bread is bought in 20% of all transactions.
- **Confidence:** Confidence assesses the likelihood that an item Y is purchased when item X is purchased. It provides insight into the strength of the association between two items. Confidence tells us how often items go together i.e If bread is bought, butter is bought 75% of the time.
- **Lift:** Lift evaluates how much more likely two items are to be purchased together compared to being purchased independently. A lift greater than 1 suggests a strong positive association. Lift shows how strong the connection is between items. Like Bread and butter are much more likely to be bought together than by chance.



Example

Lets understand the concept of apriori Algorithm with the help of an example. Consider the following dataset and we will find frequent Item-Sets and generate association rules for them:

Transaction ID	Items Bought
T1	Bread, Butter, Milk
T2	Bread, Butter
T3	Bread, Milk
T4	Butter, Milk
T5	Bread, Milk

Step 1 : Setting the parameters

- **Minimum Support Threshold:** 50% (item must appear in at least 3/5 transactions). This threshold is formulated from this formula:

$$\text{Support}(A) = \frac{\text{Number of transactions containing itemset } A}{\text{Total number of transactions}}$$

- **Minimum Confidence Threshold:** 70% (You can change the value of parameters as per the use case and problem statement). This threshold is formulated from this formula:

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Step 2: Find Frequent 1-Item-Sets

Lets count how many transactions include each item in the dataset (calculating the frequency of each item).

Item	Support Count	Support %
Bread	4	80%
Butter	3	60%
Milk	4	80%



All items have support% $\geq 50\%$, so they qualify as frequent 1-Item-Sets. if any item has support% $< 50\%$, It will be omitted out from the frequent 1- Item-Sets.

Step 3: Generate Candidate 2-Item-Sets

Combine the frequent 1-Item-Sets into pairs and calculate their support. For this use case we will get 3 item pairs (bread,butter) , (bread,ilk) and (butter,milk) and will calculate the support similar to step 2

Item Pair	Support Count	Support %
Bread, Butter	4	80%
Bread, Milk	3	60%
Butter, Milk	4	80%

Frequent 2-Item-Sets: {Bread, Milk} meet the 50% threshold but {Butter, Milk} and {Bread ,Butter} doesn't meet the threshold, so will be committed out.

Step 4: Generate Candidate 3-Item-Sets

Combine the frequent 2-Item-Sets into groups of 3 and calculate their support. For the triplet we have only got one case i.e {bread,butter,milk} and we will calculate the support.

Item Triplet	Support Count	Support %
Bread, Butter, Milk	1	20%

Since this does not meet the 50% threshold, there are no frequent 3-Item-Sets.

Step 5: Generate Association Rules

Now we generate rules from the frequent Item-Sets and calculate confidence.

Rule 1: If Bread -> Butter (if customer buys bread, the customer will buy butter also)

- Support of {Bread, Butter} = 2.
- Support of {Bread} = 4.
- Confidence = $2/4 = 50\%$ (Failed threshold).

Rule 2: If Butter -> Bread (if customer buys butter, the customer will buy bread also)

- Support of {Bread, Butter} = 2.



- Support of {Butter} = 3.
- Confidence = $2/3 = 66.67\%$ (Passes threshold).

Rule 3: If Bread $\rightarrow$ Milk (if customer buys bread, the customer will buy milk also)

- Support of {Bread, Milk} = 3.
- Support of {Bread} = 4.
- Confidence = $3/4 = 75\%$ (Passes threshold).

The Apriori Algorithm, as demonstrated in the bread-butter example, is widely used in modern startups like Zomato, Swiggy and other food delivery platforms. These companies use it to perform market basket analysis which helps them identify customer behaviour patterns and optimise recommendations.

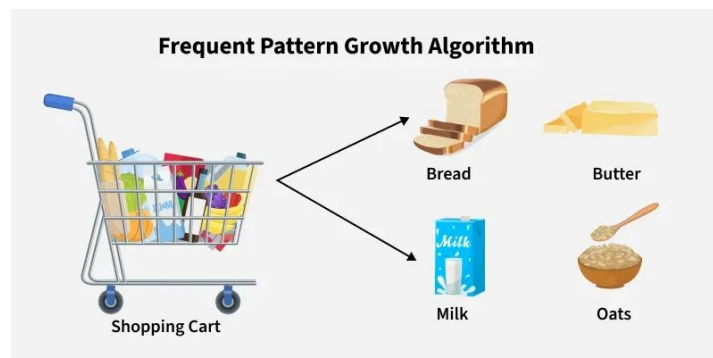
Applications of Apriori Algorithm

Below are some applications of Apriori algorithm used in today's companies and startups

1. **E-commerce:** Used to recommend products that are often bought together like laptop + laptop bag, increasing sales.
2. **Food Delivery Services:** Identifies popular combos such as burger + fries to offer combo deals to customers.
3. **Streaming Services:** Recommends related movies or shows based on what users often watch together like action + superhero movies.
4. **Financial Services:** Analyzes spending habits to suggest personalised offers such as credit card deals based on frequent purchases.
5. **Travel & Hospitality:** Creates travel packages like flight + hotel by finding commonly purchased services together.
6. **Health & Fitness:** Suggests workout plans or supplements based on users past activities like protein shakes + workouts.

4.3.2 Frequent Pattern Growth Algorithm

The FP-Growth (Frequent Pattern Growth) algorithm efficiently mines frequent itemsets from large transactional datasets. Unlike the Apriori algorithm which suffers from high computational cost due to candidate generation and multiple database scans. FP-Growth avoids these inefficiencies by compressing the data into an FP-Tree (Frequent Pattern Tree) and extracts patterns directly from it.



How FP-Growth Works

Here's how it works in simple terms:

1. **Data Compression:** First FP-Growth compresses the dataset into a smaller structure called the Frequent Pattern Tree (FP-Tree). This tree stores information about item sets (collections of items) and their frequencies without need to generate candidate sets like Apriori does.
2. **Mining the Tree:** The algorithm then examines this tree to identify patterns that appear frequently based on a minimum support threshold. It does this by breaking the tree down into smaller "conditional" trees for each item making the process more efficient.
3. **Generating Patterns:** Once the tree is built and analyzed the algorithm generates the frequent patterns (itemsets) and the rules that describe relationships between items.

Imagine you're organizing a party and want to know popular food combinations without asking every guest repeatedly.

1. List food items each guest brought transactions.
2. Count items and remove infrequent ones filter by support.
3. Group items in order of popularity and create a tree where paths represent common combinations.
4. Instead of repeatedly asking guests you explore this tree to discover patterns. For example, you might find that pizza and pasta often come together or that cake and pasta are also a common pair.

This is exactly how FP-Growth finds frequent patterns efficiently.



Working of FP- Growth Algorithm

Problem Statement: Consider a small grocery store transaction dataset. Each entry shows the set of items purchased together by a customer:

Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

We apply the FP-Growth algorithm to identify frequent itemsets (groups of items frequently bought together), using a minimum support count of 2.

Step 1: Compute Item Frequencies

Scan the entire dataset one time to determine how often each item appears.

Item	Support Count
Milk	3
Bread	4
Butter	3

All items meet the minimum support threshold (≥ 2), so none are removed.

Step 2: Order Items in Each Transaction by Frequency

Next, sort every transaction in descending order based on how frequently the items appear overall.



Transaction ID	Sorted Items by Frequency
T1	Bread, Milk, Butter
T2	Bread , Butter
T3	Bread, Milk
T4	Milk , Butter
T5	Bread

Sorting by frequency helps group common items together, allowing the FP-tree to be compressed efficiently.

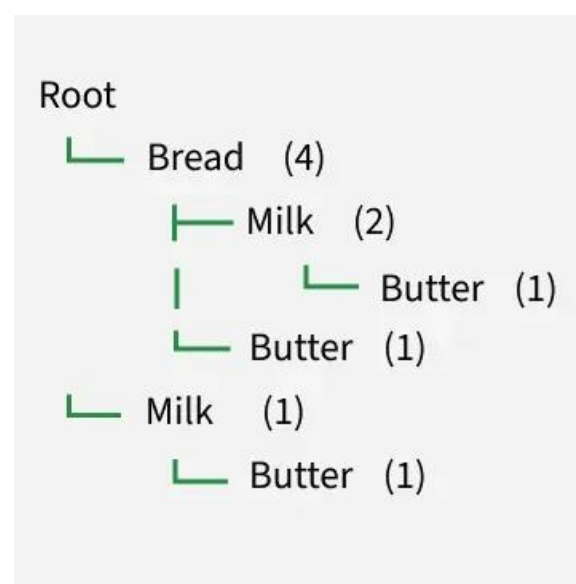
Step 3: Construct the FP-Tree

Insert the sorted transactions one by one into the FP-tree, merging any shared prefixes.

FP-Tree Insertions:

- **From T1:** Bread → Milk → Butter
- **From T2:** Bread → Butter
- **From T3:** Bread → Milk
- **From T4:** Milk → Butter (new branch since it doesn't begin with Bread)
- **From T5:** Bread

Visual FP-Tree:



Each node shows: Item (Count)



Step 4: Determine Conditional Pattern Bases

A conditional pattern base contains all prefix paths leading to a specific item. Let's examine the paths ending with Butter.

Paths that end with Butter:

- Bread → Milk → Butter (1 occurrence)
- Bread → Butter (1 occurrence)
- Milk → Butter (1 occurrence)

Thus, the conditional pattern base for Butter is:

[(Bread, Milk): 1, (Bread): 1, (Milk): 1]

Step 5: Build Conditional FP-Trees

Using the conditional pattern base, construct a smaller FP-tree for each item to identify frequent patterns involving that item. Butter's conditional FP-tree input:

- (Bread, Milk): 1
- (Bread): 1
- (Milk): 1

Count all items:

- Bread: 2
- Milk: 2

Since both meet the support threshold (≥ 2), we can now generate frequent patterns:

- {Butter, Bread}
- {Butter, Milk}
- {Butter, Bread, Milk}

Repeat the process for Milk and Bread as needed.

Step 6: Extract All Frequent Itemsets

From the FP-tree and conditional trees, we get these frequent itemsets:

- {Bread}
- {Milk}
- {Butter}



- {Bread, Milk}
- {Bread, Butter}
- {Milk, Butter}
- {Bread, Milk, Butter}

All of these appear at least 2 times in the transactions.

Summary Table

Frequent Itemset	Support Count
{Bread}	4
{Milk}	3
{Butter}	3
{Bread, Milk}	2
{Bread , Butter}	2
{Milk , Butter}	2
{Bread , Milk , Butter}	1 ✗ Not included (support < 2)

Why FP-Growth is Efficient

- It scans the database only twice.
- It avoids generating all combinations of items.
- It stores data in a compact tree, reducing redundancy.
- It uses conditional trees to mine deeper patterns efficiently.

Applications

Let's see the various applications,

- **Market Basket Analysis:** Identifying items that are frequently purchased together.
- **Recommendation Systems:** Suggesting products based on frequent item patterns.
- **Customer Behavior Analysis:** Understanding buying habits in retail and e-commerce.
- **Web Usage Mining:** Finding common browsing or click paths among users.



- **Text Mining:** Detecting frequently occurring words or phrases in documents.

4.4 Evaluation metrics for Clustering and Association Rule Mining

4.4.1 Silhouette Coefficient (Clustering Evaluation)

The **Silhouette Coefficient** is a metric used to evaluate the quality of clustering. It measures how well each data point fits within its cluster compared to other clusters. It is mainly used with clustering algorithms like **K-means clustering**, **Hierarchical clustering**, and **DBSCAN**.

Formula

For each data point:

$$S = \frac{b - a}{\max(a, b)}$$

Where

- **a** = Average distance between the point and other points in the same cluster (intra-cluster distance)
- **b** = Average distance between the point and points in the nearest neighboring cluster (inter-cluster distance)
- **S** = Silhouette score of the point

Range of Silhouette Score

$$-1 \leq S \leq 1$$

S $\approx$ **+1** $\rightarrow$ Well clustered

S $\approx$ **0** $\rightarrow$ On cluster boundary

S $<$ **0** $\rightarrow$ Misclassified

Score Value	Meaning
+1	Perfect clustering (well matched to own cluster)
0	On the boundary between clusters
Negative	Possibly assigned to wrong cluster



Interpretation

- **S close to 1** → Clusters are well separated
- **S close to 0** → Clusters overlap
- **S less than 0** → Misclassification likely

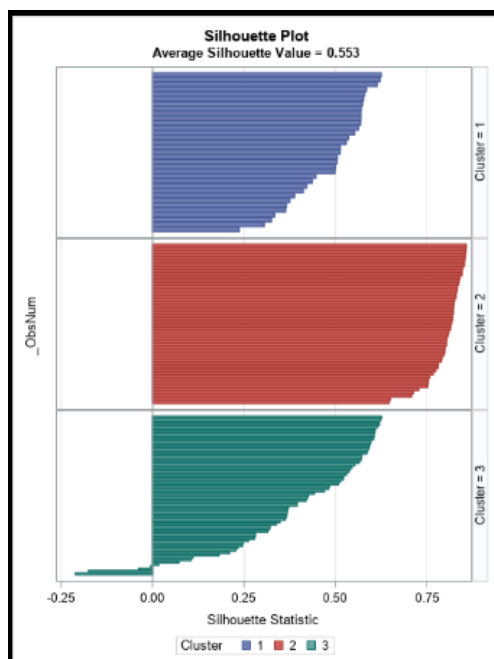
Example

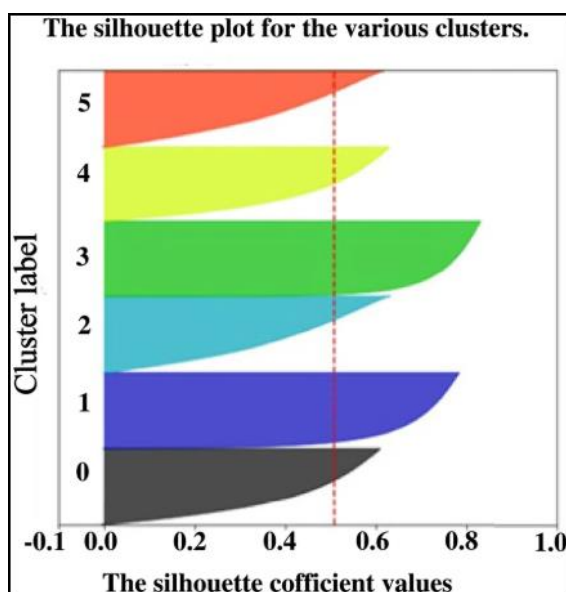
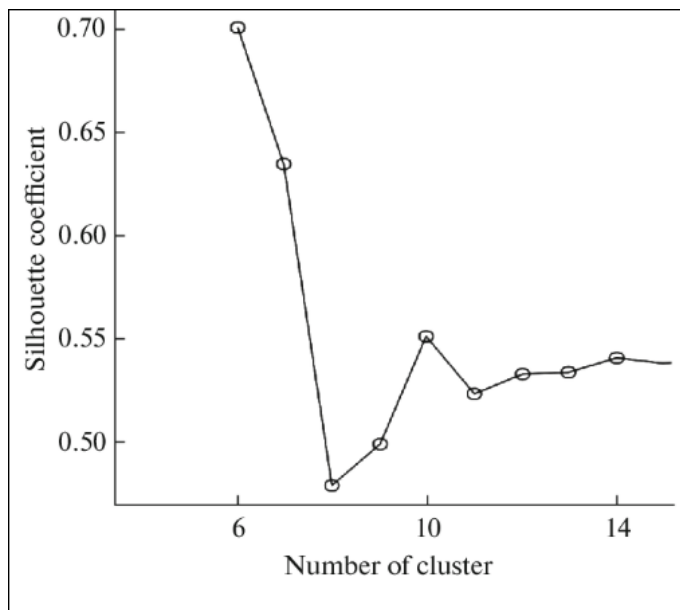
Suppose a data point has:

- $a = 2$
- $b = 6$

$$S = \frac{6 - 2}{6} = \frac{4}{6} = 0.67$$

Since **0.67 is close to 1**, the clustering is good for that point.





In a silhouette plot:

- Each bar represents one data point.
- The width shows its silhouette score.
- The average silhouette score helps choose the optimal number of clusters.

4.4.2 Support, Confidence, and Lift (Association Rule Mining)

These three metrics are used in Association Rule Mining (e.g., Apriori algorithm) to find interesting relationships in transactional data (Market Basket Analysis)



Support

Support measures the frequency of an itemset in the dataset. It helps filter out infrequent rules

Formula:

$$\text{Support}(A \rightarrow B) = \frac{\text{Transactions containing } (A \cup B)}{\text{Total transactions}}.$$

Example

Given Transaction Data

Transaction ID	Items Purchased
T1	Milk, Bread
T2	Milk, Butter
T3	Bread, Butter
T4	Milk, Bread, Butter
T5	Milk, Bread

Find Support for Rule:

Milk $\rightarrow$ Bread

Here:

- **A = Milk**
- **B = Bread**
- **A $\cup$ B = {Milk, Bread}**

Step 1: Count transactions containing Milk AND Bread

Transactions with both Milk & Bread:

- T1
- T4
- T5

Number of transactions containing (Milk $\cup$ Bread) = 3



Step 2: Total transactions

Total transactions = 5

Step 3: Apply Formula

$$\text{Support}(Milk \rightarrow Bread) = \frac{3}{5} = 0.6$$

Final Answer

Support=0.6 or 60%

That is 60% of all transactions contain both **Milk and Bread**.

Confidence

Formula

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

Step 1: Support(A ∪ B)

Transactions with Milk & Bread = 3

$$\text{Support}(Milk \cup Bread) = \frac{3}{5} = 0.6$$

Step 2: Support(A) → Milk

Transactions containing Milk:

- T1
- T2
- T4
- T5



Total = 4

$$\text{Support}(\text{Milk}) = \frac{4}{5} = 0.8$$

Step 3: Apply Formula

$$\text{Confidence}(\text{Milk} \rightarrow \text{Bread}) = \frac{0.6}{0.8} = 0.75$$

Confidence = 0.75 or 75%

If a customer buys Milk, there is a **75% chance** they also buy Bread.

Lift

Formula

$$\text{Lift}(A \rightarrow B) = \frac{\text{Confidence}(A \rightarrow B)}{\text{Support}(B)}$$

Step 1: Support(B) → Bread

Transactions containing Bread:

- T1
- T3
- T4
- T5

Total = 4

$$\text{Support}(\text{Bread}) = \frac{4}{5} = 0.8$$

Step 2: Apply Formula



$$Lift = \frac{0.75}{0.8} = 0.9375$$

Lift = **0.94 (approx)**

Support = 60% → Rule appears frequently.

Confidence = 75% → Strong rule.

Lift = 0.94 (< 1) → Milk and Bread are slightly negatively associated (not strongly related).